

COMP3320/6464 High-Performance Scientific Computing

Dr Haibo Zhang
School of Computing
Australian National University



Australian
National
University

ANU College of
Engineering & Computer Science

Assignment 2 & Final Exam

- Assignment 2 is due this Wednesday (Oct. 23) at 5 PM
- Final Exam will be held on Tuesday, November 12 starting at 2 PM in Haydon-Allen G53

COMP3320/6464 High Performance Scientific Computation 2024

- Time allowed: 3 hours and 15 minutes (including reading time)
- Reading time 15 minutes
- Permitted materials: open-book exam
- Exam questions total 100 marks
- All students answer all questions
- Clarity and conciseness in answers will be highly valued
- Marks may be lost for supplying irrelevant information
- Please write as neatly as possible
- Please ask for extra writing material if you run out of room or need to redo your answer to a question.

Structure of the Final Exam



The final exam contains 5 sections

- Short questions - 20 marks
- Four sections - each worth 20 marks - with more specific questions requiring more detailed answers and/or problem solving
 - High Performance Computer Architecture (serial and parallel)
 - Application of High-Performance Computing
 - Shared Memory Programming with OpenMP
 - SIMD parallelism, SSE, OpenMP SIMD
 - Scaling and performance modelling and analysis.
- **The lecture on photonic computing is NOT in the exam scope**

Topics for Short Questions



- COMP3320/6464 Part I: Floating-point representation and arithmetic
- COMP3320/6464 Part I: Errors (rounding, truncation, cancellation, approximation, ect.)
- COMP3320/6464 Part I: Instruction Level Parallelism (Superscalarity, Pipelining, Out-of-order execution)
- COMP3320/6464 Part I: Caching
- COMP3320/6464 Part I: Optimization of (serial) Program Performance
- COMP3320/6464 Part II: Motivation for Parallel Computers (Dennard Scaling, Moore's Law, Limitations of ILP)
- COMP3320/6464 Part II: Process vs Thread Parallelism; Thread Memory and Execution Models
- COMP3320/6464 Part II: Thread Synchronization (state diagrams, critical section, semaphores, mutexes & locks)
- COMP3320/6464 Part II: OpenMP
- COMP3320/6464 Part II: Cache Coherence
- COMP3320/6464 Part II: Scaling and Roofline

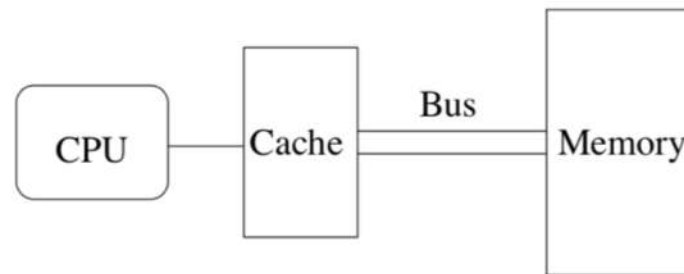
Examples of short questions

- a) Each of the following statements are either true or false. Which are they? Give a few sentences to explain each of your answers.
- i. Between the underflow limit and overflow limit there exist groups of floating-point numbers that are distributed uniformly within the group
[2 marks]
 - ii. If two real numbers are exactly representable as floating-point numbers, then the result of their floating-point sum will always be an exactly representable floating-point number, provided that the result is not larger/smaller than the overflow/underflow limit.
[2 marks]
 - iii. All floating-point operations have rounding error.
[2 marks]



- Determine performance of codes on a given architecture: caching, pipelining, superscalarity, counting FLOPs and MOPs
- Cache Coherence Protocols

(b) Consider the following memory system:



- The cache line size is 4 words
- The cache uses a write allocate policy, i.e. on a write miss the relevant cache line worth of data must first be read from memory into the cache before the write can proceed.
- The cache is write through, i.e. the store of a word propagates that change through to main memory. (Only the word is propagated, not the entire cache line).
- The bus can transfer data to or from memory at a rate of 10^6 words/sec. (Only in one direction at any given time.)

The following behaviour is observed when running a certain application.

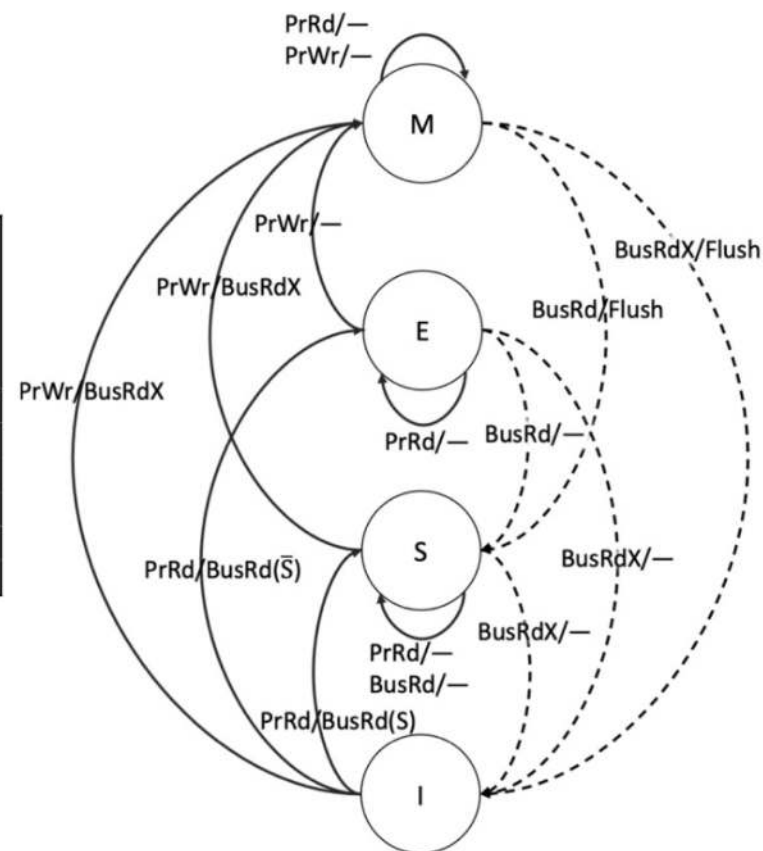
- The CPU sends memory requests (loads or stores of one word) to the cache at a rate of 10^6 requests/sec, with 25% of the requests being writes.
 - 90% of all memory requests issued by the CPU are resolved by data already in the cache (i.e. the cache hit ratio is 90%).
- (i) Given the above compute the bus traffic generated (in words/sec) by the following events:
- Read hits.
 - Read misses.
 - Write hits.
 - Write misses.
- (ii) What percentage of the total available bus bandwidth (10^6 words/sec) is used?
- (iii) What additional information do you require in order to estimate the percentage of the total available bus bandwidth used if the cache were write back?

[10 marks]

HPCA: Examples



processor action	P1 cache miss/hit, X value	P2 cache miss/hit, X value	Bus transaction (processor action initiated)	Bus snooper transaction	C1 state	C2 state	mem[&X]
No-Op	N/A	N/A	—	—	I	I	0
P1Rd X	miss, 0	N/A	BusRd($\bar{S}$)	—			
P1Wr X = 4							
P2Rd X							
P1Wr X = X*2							
P2Wr X = X-3							





- Given an algorithm you are asked questions on its performance and to propose and discuss optimizations. This may include SIMD/MIMD parallelization.

AHPC: Examples



The C pseudo-code below computes a matrix vector product of a $M \times N$ matrix (`mat`) with a vector (`vin`) of length N . The code assumes that the elements of the array `vout`, which contains components of the output vector, are initialized to zero.

```
for(int i = 0; i < M; ++i){
    for(int j = 0; j < N; ++j){
        vout[i] += mat[i][j]*vin[j];
    }
}
```

- a) Outline how you would write a routine to implement this operation using SSE2 instructions. You are not required to reproduce the SSE2 syntax exactly, but you must make your intention clear. Assume single-precision floating-point data type and explicitly write how you would allocate and structure the `vout`, `mat`, and `vin` arrays for aligned access and best vector performance.

[8 marks]

- b) Write a routine to implement this operation using OpenMP SIMD directives.

[4 marks]

- c) Assume now that in your OpenMP program the `vout` and `vin` arrays are 8-byte aligned. Discuss how the compiler may vectorize your code (there is more than one case), explicitly accounting for cases when N or $N-1$ is divisible by the SIMD vector length.

[6 marks]

Shared Memory Programming with OpenMP



Australian
National
University

- Given a serial code, write the parallel version using OpenMP and discuss it.
- Given an OpenMP code, explain its execution, potentially rewrite the algorithm for correctness or better performance.

- (a) Consider three distinct $N \times N$ matrices, A, B and C. The following code performs a matrix multiplication using a single thread of execution.

```
for ( I = 0 ; I < N ; I++)
    for ( J = 0 ; J < N ; J++) C[I][J] = 0;

for ( I = 0 ; I < N ; I++)
    for ( J = 0 ; J < N ; J++)
        for ( K = 0 ; K < N ; K++) C[I][J] += A[I][K]*B[K][J] ;
```

Two attempts to develop an OpenMP parallel implementation are made:

Parallel Version #1

```
#pragma omp parallel
{
    #pragma omp for nowait
    for ( I = 0 ; I < N ; I++)
        for ( J = 0 ; J < N ; J++) C[I][J] = 0;

    #pragma omp for
    for ( I = 0 ; I < N ; I++)
        for ( J = 0 ; J < N ; J++)
            for ( K = 0 ; K < N ; K++) C[I][J] += A[I][K]*B[K][J] ;
}
```

Parallel Version #2

```
#pragma omp parallel
{
  #pragma omp for
  for ( I = 0 ; I < N ; I++)
    for ( J = 0 ; J < N ; J++) C[I][J] = 0;

  #pragma omp for
  for ( K = 0 ; K < N ; K++)
    for ( J = 0 ; J < N ; J++)
      for ( I = 0 ; I < N ; I++) C[I][J] += A[I][K]*B[K][J] ;
}
```

You test both implementations using multiple threads on a variety of different computer systems. You find that under some circumstances both versions give **incorrect** results.

- (i) Explain why each parallel implementation can sometimes give **correct** results, but can also sometimes give **incorrect** results. Describe the hardware/runtime circumstances where each scenario is likely to hold.

- (ii) In each case what modifications to the OpenMP directives would you make to ensure that both parallel implementations were always correct. (You should make **no** modifications to the basic loop structure, but are free to move, rewrite or add new OpenMP directives.)
- (iii) Discuss the performance of your two correct OpenMP versions. You should consider issues like the number of threads used, the problem size etc. Define, as required details of the parallel computer system you are running on.

[16 marks]

SIMD parallelism, SSE, OpenMP SIMD



Australian
National
University

- More detailed questions about SIMD parallelism.
- Discuss in detail SIMD code.
- Vectorize code using either SSE or OpenMP or both.

- (c) The following text and code is taken directly from the course text book.

“The compiler, guided by the rules of the programming language and its interpretation of the source, must make certain assumptions that may limit its ability to generate optimal machine code. The typical example arises with pointer (or reference) formal parameters in the C (and C++) language:”

```
void scale_shift(double *z, double *b, double s, int n){  
    for (int i=1; i<n; i++)  
        a[i] = s*b[i-1];  
}
```

The issue being referred to is aliasing.

- (i) Explain what aliasing is.
- (ii) Why does possible aliasing prevent the compiler from vectorizing the above loop?
- (iii) Assuming you force the compiler to vectorize the above loop. Give one example of how you might call this routine and expect it to produce identical results to when you call the same but unvectorized code, and one example of when you might call this routine and expect it to report different results. Explain the reasons behind your answers.

[8 marks]

Scaling analysis and performance modelling



Australian
National
University

- More detailed questions about scalability and performance modelling
- Propose and/or discuss a scalability model.
- Discuss a roofline plot.

Scalability & performance: Examples

- (b) The speedup (S) of an OpenMP parallel code running on N identical cores can be modelled as

$$S(N) = \frac{1}{f + (1 - f)/N + \kappa N + \lambda}$$

where f is the fraction of sequential code and κ and λ are constants.

- (i) When κ and λ are zero this equation is identical to Amdahl's law. What is the justification for the inclusion of the κ and λ terms?
- (ii) Draw a graph showing how $S(N)$ varies as a function of N . Make clear what assumptions you make regarding the values of the other quantities in the equation. Highlight the key points of interest in your graph.
- (iii) If $f = 10\%$, estimate how many threads would be required to achieve $S_{1/2}$, where $S_{1/2}$ is half the maximum possible speedup? Show how you derive your answer.

[10 marks]

Remarks on answering exam questions

- **Answers need to be written in the answer book**
- Answer the exam questions in the same order as they are listed in the exam paper.
- Clearly label your answers with the corresponding question and sub-question numbers
- Leave some white space between answers for adjacent questions.
- Write legibly
- An example for the layout of answers is given in the next slide.

Remarks on answering exam questions

Put the number for question and sub-question in separate lines

1.a

(i) XX

(ii) XX



**Add white space between
answers for adjacent questions**

2.a

(i) XX

(ii) XX

Answer questions in the same order as they are listed in the exam paper.



Go well
I wish you every success and hope
we'll meet up again sometime!